

50.003 Elements of Software Construction Project Handout

Project Groupings (updated on 15 May 2026)

Project Topics

- Industry Project 1

- Industry Project 2 (updated 29 May 2026)

- Project Meeting 1 (5%)

- Project Meeting 2 (5%)

- Project Meeting 3 (5%)

- Final Project Presentation and Reports (capped at 25%)

- What Robustness Testing should I plan?

Project Groupings (updated on 15 May 2026)

- If you would like to propose your own project idea (your start-up project), please drop the instructors an email by 28 May Thu 23:59,
 - With detailed project descriptions, key deliverables and project timeline. The instructors will decide your proposal is suitable.
 - A list of **maximum** two founding team members from the **same cohort class**. The instructors will assign the rest of team members.
 - Once you obtain the clearance from the instructors, update [this excel sheet](#).
- If you are not proposing your project, your group will be assigned by 50.003 instructors.
 - You can pick **one and only one classmate within the same cohort class** to be your buddy and update [this excel sheet](#).
 - We will try allocating both you and your buddy in the same project team.
 - You can document up your proposal by following the project topic section below.
- If you are taking the 50.003 x 50.005 CoreStack project, you can put 3 names as buddy.
 - You can document up your proposal by following the project topic section below.

Project Topics

For students who are not initiating your project, you may vote for one of the following project topics. You may check with your classmates who are proposing their projects, which will be updated in the project topics once they are approved.

Here is the list of Project Topics Supported by Industry Partners.

Industry Project 1

Title	Hotel Booking System Case Study
Industry Mentors	Mr Wilbert Aristo (wilbert.aristo@ascenda.com)
Project Context	Ascenda Loyalty provides white-labelled hotel booking platforms on behalf of banks, airlines and loyalty programs worldwide. Global customers can earn & redeem hotel night stays in these platforms using accumulated points from their spendings. Some features in these platforms include text-based autocomplete search, aggregating hotel / destination searches from multiple suppliers, storage of booking data, etc. These features could serve as a challenging project topic which tests students' knowledge on data abstraction, good software design, scalability, and security.
Project Objectives	Provide students with a real-world example of a scalable and secure software system in the tech industry. Challenge students to come up with a good software design & practice when recreating (or even improving) Ascenda's current features.
Project Deliverables	Simple web application & backend API server, security test cases, reports
Main form of expected project deliverables	A web application
Crucial properties to test for the delivered software	Functionality, performance and security
Additional related resources	Product overview https://www.ascendaloyalty.com/travel-rewards , detailed writeup on case study (features to be implemented and their details) is ready and can be shared through private google docs upon request. https://docs.google.com/document/d/1on8n1vYThapSIQAqC93wsbAQoGy2fpL/
Announcement	Common Briefing (TBD) Briefing Handout FAQ

Industry Project 2 (updated 29 May 2026)

Title	DAS Individualised AI-Based Learning System: D.I.A.L
Industry Mentors	Ms Soofrina Binte Mubarak <soofrina@das.org.sg>
Project Context	Dyslexia Association Singapore (DAS), began as a community project in 1989, raising dyslexia awareness through public forums. Inspired by the forums' success, a local dyslexia association was established in April 1991, officially registered as DAS in October 1991. The mission is empowering those who learn differently, including those with dyslexia, to achieve their true potential.
Project Objectives	In this project, DAS aims to create an AI-powered software to efficiently achieve individualisation and differentiation, The mega project suite is named DAS Individualised AI-Based Learning System: D.I.A.L. In this project we focus on the following sub-systems 1. DAS Learning Screening Engine 2. DAS Cognitive Profiling System 3. DAS Adaptive Learning Activity Generator 4. DAS Error Pattern Analyzer 5. DAS Progress Monitoring System 6. DAS Intervention Recommendation Engine 7. DAS Parent Insight Dashboard Students are advised to provide solutions to AT LEAST TWO out of seven sub systems. Preferably on a subset of the subsystems that are strongly connected.
Project Deliverables	Refer to the attached sub project problem statements.
Main form of expected project deliverables	Refer to the attached sub project problem statements
Crucial properties to test for the delivered software	Functionality, Performance, Security
Additional related resources	https://das.org.sg/

	Common briefing Time: Jun 3, 2026 02:30 PM Singapore https://us06web.zoom.us/j/84097080069
--	---

Project Meeting 1 (5%)

- At least five use cases of your product 2%
- Clarification of doubts or gaps in the requirement 1%
- Development Process, Constraints and risks 1%
- Project timeline and distribution of workloads 1%

Item	Poor	Fair	Excellent
Documentation of Use Cases. Are use cases clearly defined?	Use cases are not provided. (0)	Use cases are incoherently described (i.e. does not match the objective of the software product / tool). Not enough use cases. (1)	Complete and Comprehensive description of use cases (at least five use cases). Although not mandatory at this stage, but it is good if each use case is described in the table format discussed in the class. (2) [OBJ]
Clarification of requirements. An excerpt/summary of your discussion with the client. If you are proposing your own project, then a summary of discussion with the instructor.	No summary. (0)		Provides a concise and coherent summary of discussion with the client. (1)
Software development process and risk analysis. What type of software development process you foresee to follow (agile, incremental,	No discussion. (0)	Incomplete / incoherent / missing description for some of the points. For Instance, software development process is discussed, yet risks are not well thought of and discussed. (0.5) [OBJ]	Complete and comprehensive description of potential software development process. Risks are well described. (1)

prototyping, a combination of them etc.) and what are the risks (e.g. inadequate knowledge about some tool, unknown programming language etc.) and how to mitigate the risk factors?			
Project Timeline. What are the deliverables in each of the project meeting (Project meeting 2, 3, 4 and the final presentation, it can be an estimate at this stage and may change as the project progresses)?	No discussion. (0)	Deliverables are not well thought of and/or not all deliverables for the project meetings are discussed. (0.5)	Complete and comprehensive description of all deliverables for each project meeting, starting from project meeting 2. (1)

Project Meeting 2 (5%)

- Any changes in requirement (compared to 1st meeting) 1%
- Formal documentation of use cases 1%
- Initial design (at least class diagrams and sequence diagrams of some essential use cases) 1%
- Implementation of some basic features (demo) 1%
- Feature progress records to show workload distribution 1%

Item	Poor	Fair	Excellent
Changes in Requirement. (This is shown in the report) Is there any refinement / change in the requirement since the last project meeting?	Changes are not discussed. (0)		Changes are clearly identified and discussed in the report. If there are no changes since meeting 1, then please mention that there are no changes. For external projects, include a summary of discussion (if any) that happened since project meeting 1. (1)
Documentation of Use Cases. (This is shown in the report)	No use case. (0)	Incomplete and / or inconsistent use case diagram with respect to the project requirement. (0.5)	Complete and comprehensive use case diagram and consistent with the project requirement. If you have already drawn your use case diagram in the previous meeting, you can simply include the same if nothing had changed between project meetings. (1)
Initial Design (This is shown in the report)	No class diagram. (0)	Incomplete class diagram. Does not show all the multiplicities or associations. Does not show (comprehensively) what are the operations in the class or the class diagram is	Complete and comprehensive class diagram which is consistent with the use case diagrams. Additionally, sequence diagrams are modelled to show some Important workflows. (1)

		inconsistent with the use case diagrams. (0.25)	
		Class diagram is comprehensive and consistent with the use case diagram. However, no sequence diagram is modelled to capture the critical workflows. (0.5)	
Implementation (This is shown in the meeting, no item is required for the report) Implementation of “basic” features. Snapshots and pictures are not considered demos. In general, the demonstration should be due to actual coding (Note: “Basic” features are hard to quantify. We do not penalise at this stage due to too few features implemented. What we pay attention is that coding was involved, and the features are consistent with the requirement and design)	No demonstration. (0)	Demonstration of basic features is shown; however, it is inconsistent with the requirement and is not well connected with the use-case diagram/design. The demonstration of features is buggy (e.g., crashes) or the demonstration was not due to actual coding. (0.5)	Demonstration of basic features without error and the demonstration is consistent with the use case diagram / design. (1) At least one of either frontend or backend features can be shown, e.g. making API call with an expected output, submit form to update database, user page UI, etc.
Feature progress records to show workload distribution	No record shown (0)		Clear and detailed records showing the workload distribution with appropriate breakdown (1) Recommended to highlight deliverables via explicit reference to documentation in submission or

			source code module to highlight your feature progress. Be prepared that mini in person individual interviews might be conducted to verify the contribution. If the feature is not implemented by the member (or without artefact proving the accountability), we have to right to award 0 for this component.
--	--	--	---

Note: Some projects may require system testing via crowdsourcing (e.g., testing with real users). If you are in doubt, then consult with an instructor and include it in your report.

Project Meeting 3 (5%)

- Any changes in requirement and design (compared to 2nd meeting) 0.5%
- complete design 0.5%
- Implementation of additional features (over last meeting) in your product (demo) 1%
- Test Plan 1%
 - Unit test cases (of some core features). Note test case in table document format, not the codes.
 - Integration test cases (of some core features, backend/frontend). Note test case in table document format, not the codes. You should describe what strategy is applied, decomposition or call graph, top-down or bottom up.
 - (Optional) End-to-end testing for some use case / user story.
 - A timeline showing how the rest of tests should be implemented and executed.
- Implementation of some tests (demo) 1%
- Feature progress records to show workload distribution 1%

Item	Poor	Fair	Excellent
Changes in Requirement. (This is shown in the report) Is there any refinement / change in the requirement since the last project meeting?	Changes are not discussed. (0)		Changes are clearly identified and discussed in the report. If there are no changes since meeting 2, then please mention that there are no changes. For external projects, include a summary of discussion (if any) that happened since project meeting 2. (0.5)
complete design	No design or no progress after project meeting 2. (0)		All diagrams are complete, e.g. complete set of use case diagram, class diagram, and sequence diagram, correctly capturing all the features. (0.5)

Implementation of additional features (over last meeting) in your product. Demonstration of features (discuss which use cases are fully / partially implemented and which use cases are not yet implemented). For example, you may briefly mention where you were during Project Meeting 2 and what additional progress you have made.	No demo or no progress after project meeting 2. The features were developed without human evaluation. (0)	Very slim progress since project meeting 2 and the plan about how to deliver in the final is unclear. Implementation is frequently nonfunctional. The features were developed with limited human evaluation. (0.5)	Demo is well prepared. Expected to have all functional features to be implemented. Existing bug or error can be pointed. Demonstration in isolation (features are not integrated) is allowed. Features are implemented with full supervision and control by the human engineer. Clear plan on Implementing the remaining features (if any). (1)
Testing Plan (This is shown in the report)	No testing plan (0)	Examples of unit testing tools, features/scenarios for unit testing. Additionally, tools are identified for system testing (e.g., UI testing for web app and mobile apps). However, no timeline is provided to carry out a detailed testing plan. (0.5)	Examples of unit testing tools, system testing tools (UI testing tools), features/scenarios for unit testing identified. Additionally, a detailed timeline is presented to carry out the test planning. (1)
Implementation of some (unit) tests (demo) Demonstration of (unit) tests. Walking through the software is not considered a test. In general, if there is no coding user study or crowdsourcing involved for the tests, then there is no test. The unit test must be written in proper test frameworks, (e.g. Jest, RSpec and equivalent). You may	No test is written / conducted (if user study) and the basic set of tests is reflected via using the software by its developers. There is test, but no coding was involved (or no user study involved) to prepare for these tests. (0)	Many tests are written, and they run. Demonstration of test execution crashes or the tests are not aligned with the project objectives / use cases. (0.5)	Many tests are well written; they run properly and are consistent with the use cases of the project. (1)

augment the test suites with other frameworks such as Postman, Cypress, Selenium and Cucumber.			
Feature progress records to show workload distribution	No record shown (0)		Clear and detailed records showing the workload distribution with appropriate breakdown (0.5) Recommended to highlight deliverables via explicit reference to documentation in submission or source code module to highlight your feature progress. Be prepared that mini in person individual interviews might be conducted to verify the contribution. If the feature is not implemented by the member (or without artefact proving the accountability), we have to right to award 0 for this component. Everyone must contribute. Individual penalty for those who don't.

Final Project Presentation and Reports (capped at 25%)

- Group Report and code repo (15% + 1% bonus)
 - Requirement (2%)
 - Design (3%)
 - Implementation Challenges (2%)
 - Testing
 - Unit testing backend and frontend (3%)
 - Integration testing (2%)
 - System end-to-end testing (1%)
 - Robustness testing (1%)
 - Feature progress records to show workload distribution (1%)
 - For industry-backed projects, please make sure
 - you obtain written approval from your industry mentors that your project submission includes codes, data samples and other IPs belonging to the industry partner's business.
 - Please discuss impact of your project on sustainability, as well as any consideration for diversity and inclusion (e.g., different cultures, demographic groups, etc.). You may discuss how your project contributes to the United Nation SDG: <https://sdgs.un.org/goals> (Bonus mark 1%)
- Individual Report & Peer Review (5%)
 - Your report should consist of the following sections:
 - Contribution to requirement formula on and refinement
 - Contribution to the design
 - Contribution to the implementation
 - Clearly articulate which subsystems were implemented by you
 - Contribution to testing
 - Clearly articulate which type of tests were designed and developed by you
 - An AI Hallucination diary.
 - Reflection. If the project was not successful (in your opinion), then what is the main reason behind such failure?
 - Max 3 pages.
 - You will also need to submit your peer review evaluation.
 - If you do not submit, your individual report marks will be 0.
 - The peer evaluation results will be compiled and used to moderate the marks allocated for your individual reports.
 - If your peer evaluation result shows that you do not contribute to the project, the instructors have the right to investigate further and would moderate the group marks awarded to you.
- Final presentation (5%)
 - 15 minutes presentation that includes a demo of the product, a short video for a high-level description of the software and how it was tested.
 - The video should not be longer than 3 minutes. It serves as a backup material for your presentation in case of technical problems. The school might seek your permission to use the videos as for future events and publicity.
 - For industry-backed projects, please make sure

- you obtain written approval from your industry mentors that your project video may contain some of the information, data and IPs belonging to the industry partner's business.
 - A well-planned & executed, clear and concise presentation will help the graders to better appreciate your reports and project contributions.
- Final Report Grading Rubrics

Item	Poor	Fair	Excellent
Requirement Detailed description via use case diagram	No use case diagram. (0)	Some use case diagram, yet it is incomplete (does not cover all features) or incomprehensive or no misuse cases were modelled. (1)	Complete and comprehensive use case diagram clearly showing the different features of the product. Comprehensively model misuse cases. (2)
Design Design of different subsystems via UML diagrams	No class diagram nor sequence diagram. (0)	Class diagram is present. However, it is incomplete, does not show all the associations or multiplicities, does not show (comprehensively) what are the different operations in the class, or it is inconsistent with the use case diagram. (1)	Complete, comprehensive and consistent class diagram with respect to its use case diagram. Sequence diagrams are as comprehensive as possible to reflect the detailed usage scenarios of the product. (3)
		Class diagram is complete and consistent with use case diagram, but does not comprehensively model sequence diagrams to reflect product workflow. (2)	
Implementation Challenges Divide this section into multiple subsections 1) Algorithmic	No discussion on challenges. (0)	Challenges are outlined, yet there is no comprehensive descriptions on what measures were taken to overcome these	Clearly outlines the challenges and comprehensively describe how they were addressed or what alternative measures were

challenges, 2) Engineering challenges (e.g. usage of certain tools, integration issues etc.) 3) Testing challenges. Every project should have at least engineering and testing challenges. Algorithmic challenges might be specific to a few projects only (e.g. for Games).		challenges or these challenges were not even addressed in the project. (1)	taken. (2)
Unit Testing Implementation of unit tests and Demonstration of tests. Walking through the software is not considered a test. Test cases must be written and implemented using testing framework such as Jest, RSpec or equivalent.	No test or no progress made compared to meeting 3. There are some additional unit testing, but no coding was Involved. (0)	The frontend or backend unit testing is/are missing. Testing was done only based on the user stories, and alternative and error scenarios are not covered. Demonstration of test execution crashes or the tests are not aligned with the project objectives/use Cases. (1)	Extensive unit testing. They run properly and are consistent with the use cases of the project. Boundary tests and negative cases tests are included. (3)
Integration Test Implementation of integration tests and Demonstration of tests. Walking through the software is not considered a test. Test cases must be written and implemented using testing framework such as Jest or JUnit	No test or no progress made compared to meeting 3. No integration testing plan. There are some additional integration testings, but no coding was Involved. (0)	The frontend or backend integration testing is/are missing. Demonstration of test execution crashes or the tests are not aligned with the project Objectives/sequence diagram/use Cases. (1)	Extensive integration testing. They are reflecting faithfully the integration test plan. They run properly and are consistent with the use cases and sequence diagram of the project. (2)
System Testing &	No proper testing	Only system testing	System and

Robustness Testing How was the product tested? Clearly articulate what features were tested, what was the process followed for testing, what tools were used and what were the findings. You are encouraged to use automated end-to-end testing tools such as Cypress. For Robustness Testing refer to the session below.	or code-a-bit-test-a-bit type of testing (0)	but not testing to check the robustness. For games, both system and robustness testing can be done by game players. (1)	robustness testing is performed comprehensively and described in the report in detail. (2)
Feature progress records to show workload distribution	No record shown (0)	There are records showing the workload distribution with appropriate breakdown (0.5)	Clear and detailed records showing the workload distribution with appropriate breakdown. The progress of the project reflects there are clear project management work done in rescheduling and balancing workload. (1) Recommended to highlight deliverables via explicit reference to documentation in submission or source code module to highlight your feature progress. Be prepared that mini in person individual interviews might be conducted to verify the contribution. If the feature is not implemented by the member (or without artefact proving the accountability), we have to right to

			award 0 for this component. Everyone must contribute. Individual penalty for those who don't.
--	--	--	--

What Robustness Testing should I plan?

- Find Fuzzing targets in your project.
- Use any language/platform to implement a fuzzer.
- Ideally this fuzzer should be able to run and generate tests over a very long period (e.g., 24 hours).
- The final version of the fuzzer may not be ready by Project Meeting 3, but it should be ready by the final presentation.
- If time permitted, try
 - <https://jmeter.apache.org/>
 - <https://hypothesis.readthedocs.io/en/latest/>
 - <https://github.com/dubzzz/fast-check>